

Compléments de Programmation

Licence 1 UPEC 2025/2026

TM 5 : Encore récursivité

Exercice 1: Exercices de compréhension, à faire sur feuille.

Pour chacune des fonctions ci-dessous, cherchez à comprendre

1. est-ce que la fonction termine toujours ? jamais ? seulement certains cas ?
2. si elle termine, qu'est-ce qu'elle calcule ?

Fonction 1 :

```

1 public static int boucle1(int n){
2     if(n==0){
3         return 1;
4     }
5     return boucle1(n+1);
6 }
```

Fonction(s) 2 :

```

1 public static int boucle2(int n){
2     if (n<=0){
3         return 1;
4     }
5     else{
6         return 2*boucle3(n-1);
7     }
8 }
9 public static int boucle3(int n){
10    if(n==0){
11        return 1;
12    }
13    return boucle2(n);
14 }
```

Exercice 2: Avec des mots !

Dans une classe `Exo2` :

1. Écrivez une fonction récursive `palindrome` qui prend en paramètre une chaîne de caractères et qui renvoie `true` si le mot passé en paramètre est un palindrome, `false` sinon. Par exemple, `palindrome("radar")` et `palindrome("ressasser")` doivent renvoyer `true`, alors que `palindrome("enseignant")` doit renvoyer `false`.

Pour cela, on utilisera seulement les méthode `charAt`, `length` et `substring` de la classe `String`

2. Écrivez une fonction récursive `verlan` qui prend en paramètre une chaîne de caractères et qui renvoie la chaîne de caractères dont toutes les lettres écrites sont dans l'ordre inverse. Par exemple, `verlan("hello")` doit renvoyer `"olleh"`.

On précise que l'on peut concaténer deux chaînes de caractères en utilisant le `+` : par exemple, `"hello" + "toto"` renvoie `"hellototo"`.

Exercice 3: Algorithme d'Euclide

On rappelle que si a et b sont deux entiers positifs tels que $a \geq b \geq 0$ et $(a, b) \neq (0, 0)$, alors le pgcd de a et b vérifie :

$$\text{pgcd}(a, b) = \begin{cases} b & \text{si } a \bmod b = 0, \\ \text{pgcd}(b, a \bmod b) & \text{sinon,} \end{cases}$$

où r est le reste dans la division euclidienne de a par b , c'est à dire que $r = a \bmod b$.

Écrivez une fonction récursive `pgcd` qui prend en paramètre deux entiers positifs ou nuls a et b et qui renvoie le `pgcd` de a et de b en exploitant la propriété précédente.

Par exemple, `pgcd(2, 3)` renvoie 1 et `pgcd(42, 54)` renvoie 6.

Exercice 4: Divisions en deux

Dans une classe `Exo4` :

1. Écrivez une fonction récursive `public static void ligneSansRetour (int n, char c)` qui prend en paramètre un entier n et un caractère c et qui affiche n fois le caractère c , sans aucun retour à la ligne. Par exemple, l'appel à `ligneSansRetour(5, 'x')` suivi de `ligneSansRetour(3, 'o')` doit afficher

```
xxxxxooo
```

2. Écrivez une fonction récursive `public static void decoupage (int n, int d)` qui prend en paramètre un entier n et un entier d et qui affiche une ligne de longueur n divisée en 2^d séquences de 'x' et 'o'. Pour tester, on prendra n égal à une puissance de 2, et d égal à un nombre inférieur à cette puissance.

Par exemple, l'appel à `decoupage(16,0)` doit afficher

```
xxxxxxxxoooooooo
```

L'appel à `decoupage(16,1)` doit afficher

```
xxxxoooooooo
```

L'appel à `decoupage(16,2)` doit afficher

```
xxooxxooxxooxxoo
```

L'appel à `decoupage(16,3)` doit afficher

```
xoxoxoxoxoxoxoxo
```

3. Écrivez une fonction récursive `public static void fractale (int n, int d)` qui prend en paramètre un entier n et un entier d et qui affiche des lignes de tailles n , divisées en 1 à 2^d séquences de 'x' et 'o'. Pour tester, on prendra n égal à une puissance de 2, et d égal à un nombre inférieur à cette puissance. Par exemple, l'appel à `fractale(16,3)` doit afficher

```
xxxxxxxxoooooooo
xxxxoooooooo
xxooxxooxxooxxoo
xoxoxoxoxoxoxoxo
```

Exercice 5: Un petit train...

Téléchargez le fichier `Wagon.java` sur eprel. Il contient une classe `Wagon` avec une fonction pour créer un petit train.

Vous allez écrire :

- une méthode d'instance `toString` récursive qui renvoie une chaîne de caractère qui représente joliment un train à partir de son premier wagon
- une méthode d'instance récursive qui calcule le nombre de wagons d'un train
- une méthode d'instance récursive qui calcule le nombre total de places d'un train
- une méthode d'instance récursive qui ajoute un wagon frets à la fin du train

Exercice 6: Pavage

Téléchargez le fichier `PavageFonction.java` sur eprel.

1. En regardant uniquement le code, arrivez-vous à comprendre ce que fait la méthode `pavageSousFonction` ?
2. Exécutez la classe, faites varier le paramètre `depth` (ligne 55). Arrivez-vous à comprendre ce que fait la méthode ?
3. Ajoutez dans la classe une méthode statique

```
public void pavageSurFonction(Graphics2D g2, double xGauche, double xDroite,  
                             double yBas, double yHaut, int depth)
```

qui pave en bleu l'espace au dessus de la fonction f .
4. Modifiez la méthode `pavageSousFonction` de façon à ce qu'elle renvoie un `double` égal à la somme des aires des rectangles. À quoi pourrait servir une telle fonction ?
5. Modifiez la méthode `pavageSousFonction` de façon à ce que, au lieu de diviser les rectangles en quatre, les rectangles soient divisés en deux selon leur plus grande dimension.